

DFT and FFT

Script: Chapter 8 (discrete Fourier transform, FFT algorithms)

Learning goals

- Compute small DFTs by hand and use the DFT properties (periodicity, symmetry for real signals, Parseval).
- Distinguish frequency resolution $\Delta f = f_s/N$ (record length) from the bin spacing obtained by zero padding.
- Explain leakage and choose a window by main-lobe width, side-lobe level, scalloping loss and equivalent noise bandwidth.
- Implement an iterative radix-2 FFT and verify its $\mathcal{O}(N \log N)$ complexity against the $\mathcal{O}(N^2)$ direct DFT.
- Compute a crank-angle-triggered knock spectrum on the ESP32 and judge the memory and operation budget of an 8-bit Uno.

Background

The N -point DFT of $x[n]$, $n = 0, \dots, N-1$, and its inverse are

$$X[k] = \sum_{n=0}^{N-1} x[n] W_N^{kn}, \quad x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k] W_N^{-kn}, \quad W_N = e^{-j2\pi/N}.$$

$X[k]$ samples the DTFT of the *finite record* at $f_k = k f_s/N$. Multiplying by a window $w[n]$ before the DFT convolves the spectrum with the window transform: a sinusoid between two bins leaks into all bins (*leakage*). The radix-2 decimation-in-time FFT splits an N -point DFT into two $N/2$ -point DFTs of the even and odd samples,

$$X[k] = G[k] + W_N^k H[k], \quad X[k + N/2] = G[k] - W_N^k H[k],$$

recursively, giving $\log_2 N$ stages of $N/2$ *butterflies*.

The FFT written on this sheet is used again on Sheets 11 and 12. Its interface is fixed (file `fft.h`):

```
1 void fft(double *re, double *im, int n, int inverse);
2 /* in place, n = power of two; inverse = 1 includes the factor 1/n */
```

Exercise 8.1 DFT properties**PAPER**

- Write down the 4×4 DFT matrix and compute the DFT of $x = \{1, 2, 0, -1\}$. Check Parseval's relation $\sum |x[n]|^2 = \frac{1}{N} \sum |X[k]|^2$.
- Compute the 8-point DFT of $x = \{1, 1, 1, 1, 0, 0, 0, 0\}$ in closed form (geometric series) and give $|X[k]|$. Which symmetry do you observe and why must it hold?
- Resolution.** The knock measurement window extends from 10° to 70° after TDC, $f_s = 40$ kHz. How many samples does it contain at 3000 rpm and at 6000 rpm, and what is the frequency resolution Δf ? Can a knock component at 6.5 kHz be separated from the valve impact at 8 kHz (with a Hann window)? Compare Δf with the natural line width $B = 1/(\pi\tau)$ of the knock burst (Sheet 7).
- The window of c) at 3000 rpm is zero padded to $N = 256$. What changes in the spectrum, what does not? What *does* improve the resolution?

- e) **Windows.** Give main-lobe width (null to null, in bins) and highest side lobe for the rectangular, Hann, Hamming and Blackman windows. The knock component at 6.5 kHz is 60 dB stronger than a valve component at 8 kHz (9.6 bins away for $N = 256$). Which windows can reveal the weak component?
- f) **Complexity.** Count complex multiplications of the direct DFT and of the radix-2 FFT for $N = 256$. At 6000 rpm, how many 256-point FFTs per second does an angle-synchronous knock detection for a 4-cylinder engine need, and how many real multiplications per second is that?

Exercise 8.2 Own FFT

JSLINUX

Files: `code/jslinux/sheet08/fft.h` (template of the FFT header)
`code/jslinux/sheet08/fft_test.c` (test program)

- a) Implement the direct DFT `dft()` in `fft_test.c`. Avoid N^2 calls of `sin/cos`: tabulate W_N^m , $m = 0..N - 1$, and use the index $m = (kn) \bmod N$.
- b) Implement `fft()` in `fft.h`: bit-reversal permutation, $\log_2 N$ butterfly stages, $1/N$ scaling for the inverse. Compute the twiddle factors of a stage by the recursion $W_L^{k+1} = W_L^k W_L$ (one `sin/cos` pair per stage). The program compares FFT and DFT for random complex data, checks $\text{IFFT}(\text{FFT } x) = x$, Parseval, and conjugate symmetry. Also check your results of Exercise 8.1 a) and b).
- c) Implement `fit_exponent()` (least squares in log-log coordinates). Measure the runtime of DFT and FFT versus N and determine the exponents. Plot $t_{\text{FFT}}/(N \log_2 N)$ – is it constant? Explain deviations for small and for large N .
- d) How does the round-off error of the FFT grow with N ? Why does the twiddle recursion increase it slightly, and how could one avoid that?

Exercise 8.3 Windows, leakage and the knock spectrum

JSLINUX

Program `code/jslinux/sheet08/windows.c` uses your `fft.h`. Spectra are scaled as amplitude spectra, $A_k = 2|X[k]|/\sum w[n]$, so that a bin-centred sinusoid of amplitude A appears at $20 \log_{10} A$ dB.

- a) Implement the periodic Hann, Hamming and Blackman windows. The program computes coherent gain $\sum w/N$, equivalent noise bandwidth $\text{ENBW} = N \sum w^2/(\sum w)^2$, scalloping loss (tone exactly between two bins) and the highest side lobe. Compare with 8.1 e).
- b) Leakage: $x[n] = \sin(2\pi 6500 n/f_s) + 10^{-3} \sin(2\pi 8000 n/f_s)$, $N = 256$, $f_s = 40$ kHz. Which windows reveal the weak component? Explain the peak levels with the scalloping loss.
- c) Zero padding: two equally strong tones at 6500 Hz and 6900 Hz, records of $N = 64, 128, 256$ samples, each zero padded to 4096 points (Hann). Implement the parabolic interpolation of the peak position $\delta = \frac{1}{2} \frac{A_{k-1} - A_{k+1}}{A_{k-1} - 2A_k + A_{k+1}}$ (dB values). For which record length are the two tones resolved?
- d) Knock spectrum: record $N = 256$ samples of `es_knock()` at 40 kHz starting at the TDC of a knocking cylinder (3000 rpm, `knock_prob = 1`), i.e. 0° – 115° ATDC, and the window 10° – 68° ATDC (128 samples). Identify the peaks. Why is the knock peak higher in the shorter window although it contains less signal energy? Where is the valve impact (8 kHz, 90° ATDC)?

Exercise 8.4 FFT on the microcontroller

WOKWI

Project `code/wokwi/sheet08_knockfft/` (ESP32 + `engine-sim` chip): `KNOCK` → D34 (ADC1), `TDC` → D18, `KNK` (ground truth) → D19. Add your `fft.h` of Exercise 8.2 to the project.

- a) The sketch waits for the rising edge of the TDC pulse and then records $N = 256$ samples at $f_s = 40$ kHz. Which crank-angle range does the record cover at 1500, 3000 and 6000 rpm? Why is the analysis synchronised to the crank angle and not to time?
- b) Implement the capture with a deadline loop (`micros()`, period 25 μ s); count missed deadlines in `late` and record whether `KNK` was high during the window.
- c) Implement the Hann window, the search for the largest bin above 3 kHz and the parabolic peak interpolation. Why must the search exclude the low-frequency range? Set the knock probability to 50 % and collect about 20 windows: compare peak frequency, peak level and the knock-band energy `e_knock` (6–7 kHz) for windows with and without knock (`knk`). Plot one spectrum in JSLinux from the CSV output.
- d) **Uno.** Open `code/wokwi/sheet08_unoffft/` (synthetic test signal, same `fft.h`; on the AVR `double` is a 32-bit float). Compile for $N = 64, 128, 256$ and read the memory report. What is the largest feasible N and why? Add the operation count (butterflies, real multiplications and additions). Estimate the execution time from the operation count, assuming roughly 100–150 clock cycles per soft-float operation at 16 MHz, and compare with the time between two TDCs at 3000 rpm. Suggest two ways to fit a 256-point spectrum into the Uno.

Hint: Wokwi is not cycle-accurate: `micros()` differences (`tcap`, `tfift`) are simulated times and only indicative. The Serial output of one window (130 lines) takes about 0.25 s at 115200 baud; TDC events during printing are simply skipped.

Deliverables

DFT computations of 8.1; your `fft.h` (it is reused on Sheets 11 and 12!) with the error and runtime tables of 8.2; the window tables and the resolution experiment of 8.3; for 8.4 a CSV spectrum of one knocking and one non-knocking window with your knock/no-knock statistics, and the Uno memory table with the operation-count estimate.